

PARALLEL AND GPU COMPUTING

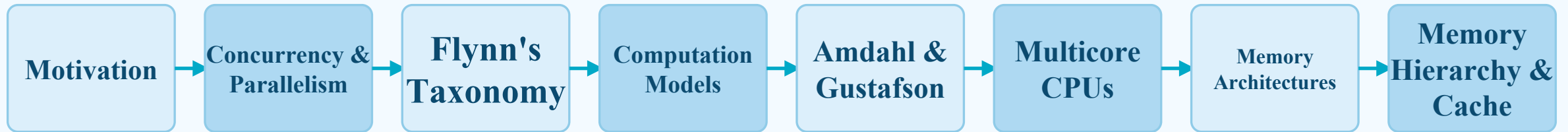
Chapter 1: Introduction

Course Code: 26ECAC304

Semester V | B.E. CSE (Artificial Intelligence)

Chapter Roadmap

A step-by-step journey through the foundations of parallel and GPU computing

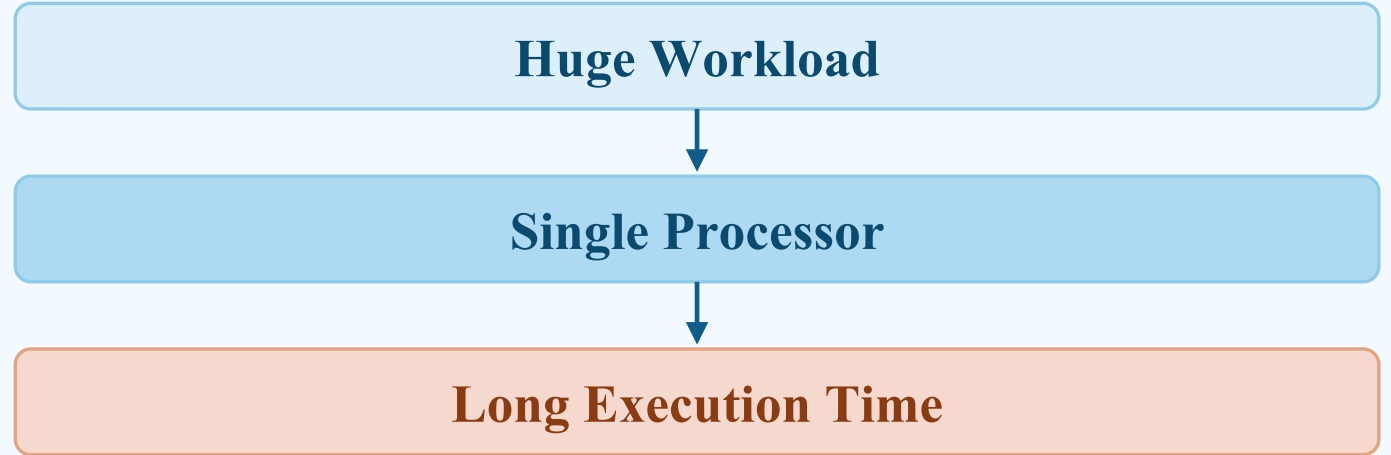


KEY TAKEAWAY Each block builds on the previous one — by the end you can classify architectures, compute theoretical speedup, and explain memory design.

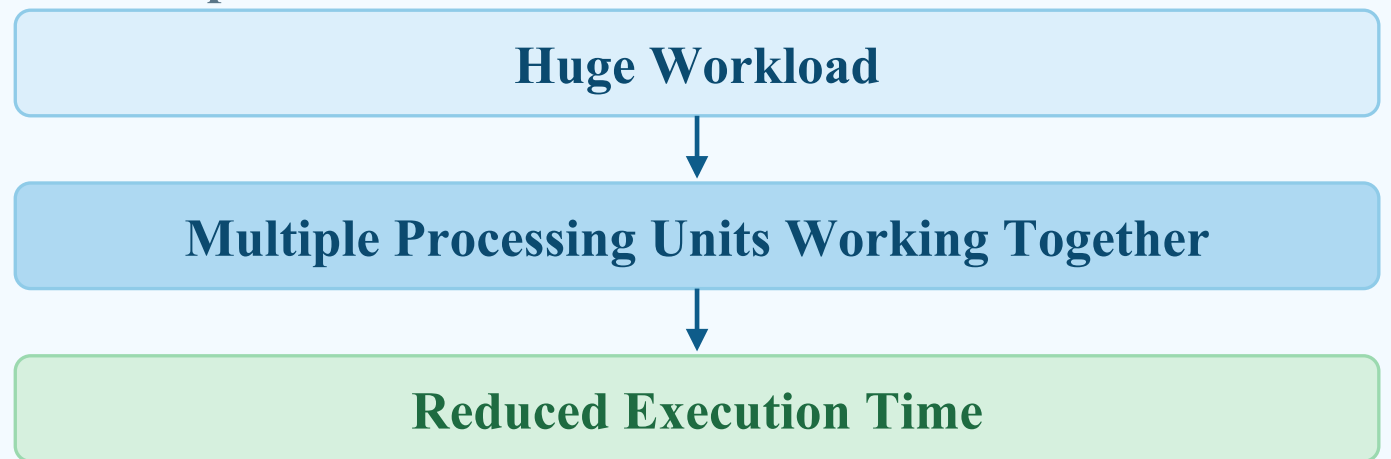
Why Do We Need Parallel Computing?

- Explosive growth in computational workloads
- AI / deep learning model training
- Image and video processing
- Real-time applications (robotics, finance, gaming)

Sequential path

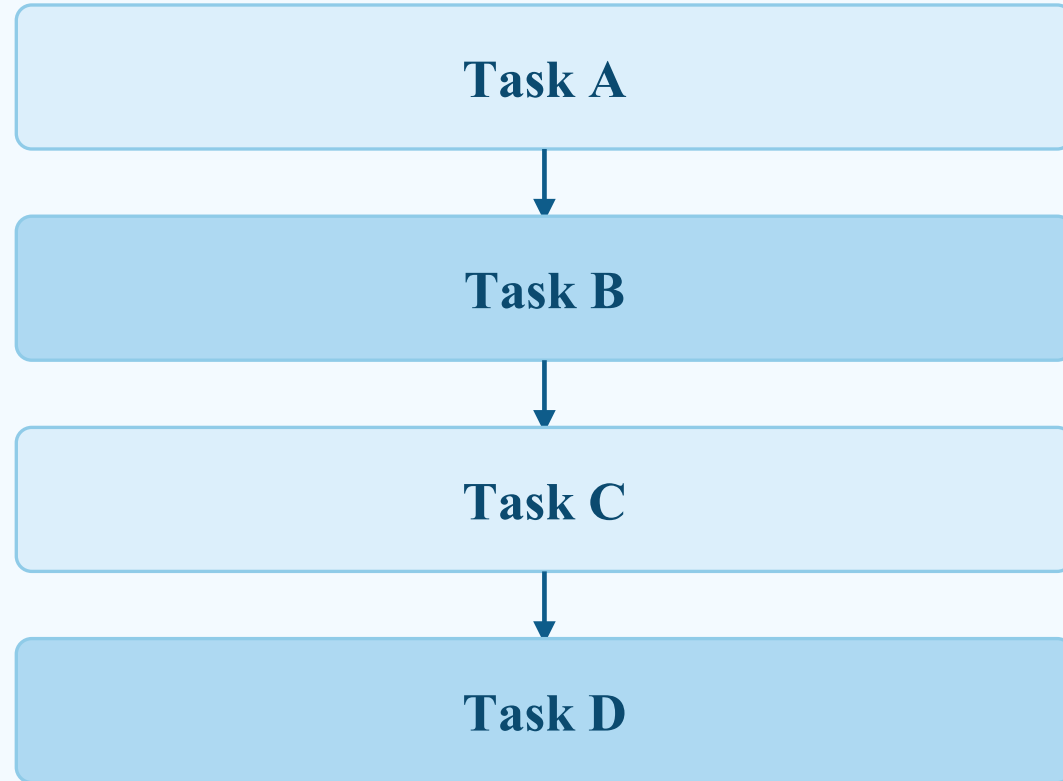


Parallel path



Sequential Computing

- A single processor/core executes one instruction (or task) at a time
- Each task must complete before the next one begins
- Simple to reason about, but does not scale with growing workloads



KEY TAKEAWAY Sequential execution is easy to write and debug, but total time = sum of all task times.

One core → one task executed at a time, strictly in order

Limitations of Single-Core Processors

1. Only one main processing unit

- A single-core CPU has only one core to execute the program's work.
- Multiple tasks cannot truly execute at the same time on that core.

2. Limited performance improvement

- We cannot keep making one core faster forever.
- Increasing processor speed has practical limits.

3. Heat generation

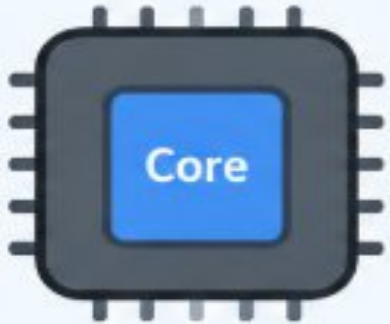
- Making the processor run faster generally increases power consumption and heat.
- Too much heat is difficult to manage.

4. High execution time for large problems

- Large calculations have to be processed through the available single core.
- As the problem becomes bigger, execution time can become very high.

From Single-Core to Multicore

Single-Core Processor (Earlier)

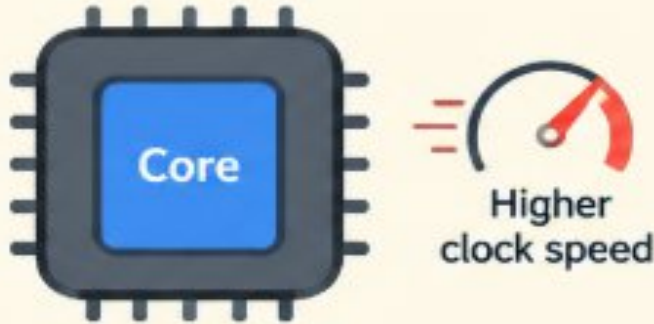


One core
(one processing unit)

- Only one main processing unit.
- Multiple tasks cannot truly execute at the same time on that core.



Faster Single-Core (Attempt to Improve)

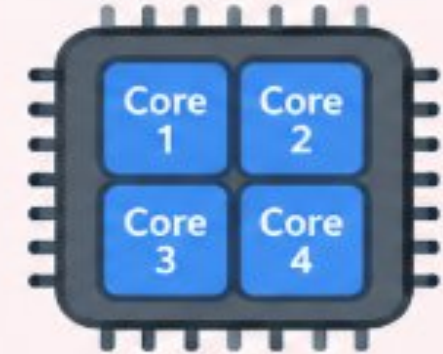


Same single core,
just faster

- Limited performance improvement.
- Increasing speed has practical limits.



Multi-Core Processor (Modern Approach)



Multiple cores
(multiple processing units)

- More than one core.
- Tasks can be executed in parallel.
- Better performance and efficiency.

What is Concurrency?

Definition

- Concurrency is dealing with multiple tasks by making progress on them at overlapping times, even if they are not executed at exactly the same time.
- Tasks may interleave on a single processor by switching back and forth

Example: Cooking a meal (one cook)

Boil water

Chop vegetables

Prepare sauce

The cook overlaps these tasks — starts water boiling, chops while it heats, checks the sauce — switching attention between them.

KEY TAKEAWAY Concurrency is about structure — managing multiple in-progress tasks — even on a single core.

Example: Concurrency

Example: Concurrent Array Processing

One CPU core, two tasks:

Task A: Add elements of Array A

Task B: Find maximum of Array B

Concurrent execution:

Time $\rightarrow$ A $\rightarrow$ A $\rightarrow$ B $\rightarrow$ B $\rightarrow$ A $\rightarrow$ B $\rightarrow$ A $\rightarrow$ B

CPU switches between Task A and Task B.

Only one task executes at a time, but both tasks make progress.

What is Parallelism?

- Multiple tasks execute simultaneously using multiple processing resources (cores/processors)

Example: Array Addition

Given:

$A = [10, 20, 30, 40]$

$B = [1, 2, 3, 4]$

Sequential:

$A[0]+B[0] \rightarrow A[1]+B[1] \rightarrow A[2]+B[2] \rightarrow A[3]+B[3]$

Parallel:

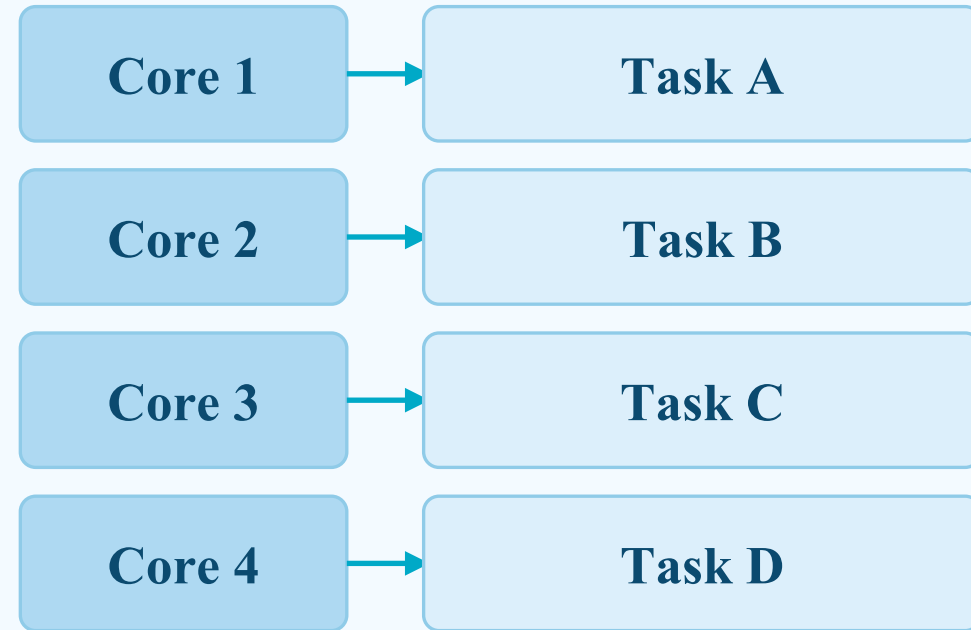
Core 1 $\rightarrow A[0]+B[0] = 11$

Core 2 $\rightarrow A[1]+B[1] = 22$

Core 3 $\rightarrow A[2]+B[2] = 33$

Core 4 $\rightarrow A[3]+B[3] = 44$

Result: $C = [11, 22, 33, 44]$



All four run at the SAME instant, on separate cores

Concurrency vs Parallelism

Feature	Concurrency	Parallelism
Meaning	Managing multiple tasks with overlapping progress	Executing multiple tasks at the same instant
Hardware requirement	Works even with a single core	Requires multiple cores/processors
Execution	Interleaved / time-shared	Simultaneous
Example	One cook switching between dishes	Four cooks working at once
Goal	Better structure	Faster raw execution

Sequential vs Concurrent vs Parallel — Timelines

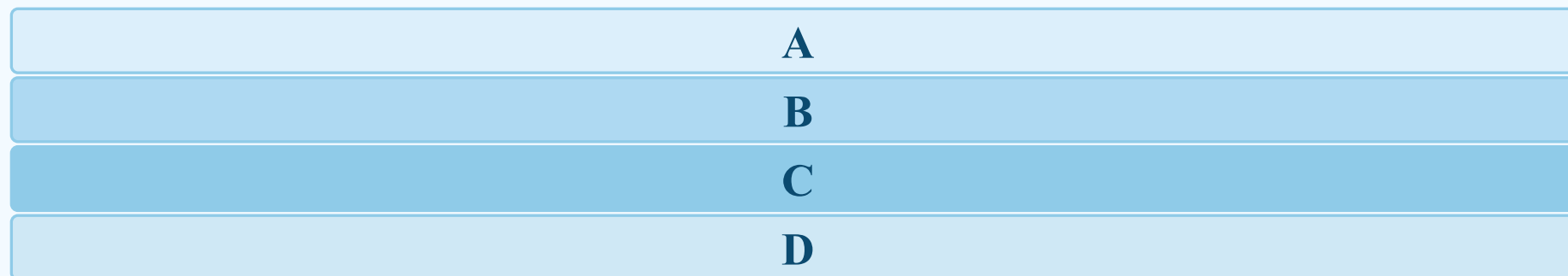
Sequential



Concurrent
(1 core, time-shared)



Parallel
(4 cores, same
instant)



time →

KEY TAKEAWAY Sequential = one at a time. Concurrent = interleaved on one core. Parallel = truly simultaneous on multiple cores.

Concurrency — Characteristics

Characteristic	Explanation
Multiple Tasks in Progress	Multiple tasks are handled during the same time period, although they may not execute simultaneously.
Can Work on a Single Core	A single CPU core can rapidly switch between tasks using time-sharing/context switching.
Independent or Dependent Tasks	Tasks may execute independently or may depend on results from other tasks.
Resource Sharing	Multiple tasks can share CPU, memory and other resources with proper coordination.

Concurrency — Limitations

Limitation	Explanation
Context-Switching Overhead	Switching between tasks requires additional processing and can reduce performance.
Synchronization Problems	Shared resources can cause race conditions if tasks are not properly synchronized.
Increased Complexity	Concurrent programs are harder to design, test, and debug because execution order can vary.

Parallelism — Characteristics

Characteristic	Explanation
Simultaneous Execution	Multiple tasks or parts of a problem execute at the same time using different processing resources.
Multiple Processing Resources	Uses multiple CPU cores, processors, or computers to perform work simultaneously.
Work Division	A large problem is divided into smaller parts that can be processed simultaneously.
Improves Performance	The main goal is to reduce execution time.
Data & Task Parallelism	Supports data parallelism (same operation on different data) and task parallelism (different tasks executed in parallel).

Parallelism — Limitations

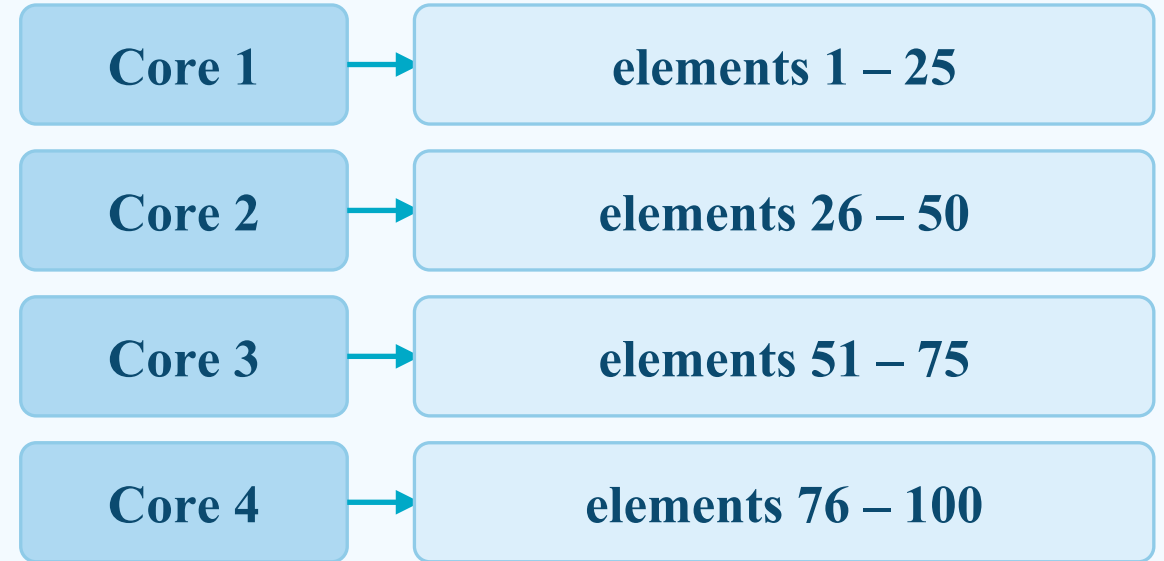
Limitation	Explanation
Not Every Problem Is Parallelizable	Problems with strong sequential dependencies cannot be fully executed in parallel.
Load Imbalance & Limited Speedup	If work is not divided equally, some cores remain idle. Therefore, adding more cores may not provide proportional speedup.

Data Parallelism

- Same operation is applied to different chunks of data, simultaneously
- Each core/thread works on its own slice of the dataset

$$A[i] = B[i] + C[i]$$

Example: element-wise array addition, split evenly across 4 cores



KEY TAKEAWAY Same instruction, different data — ideal for GPUs, which have thousands of simple cores.

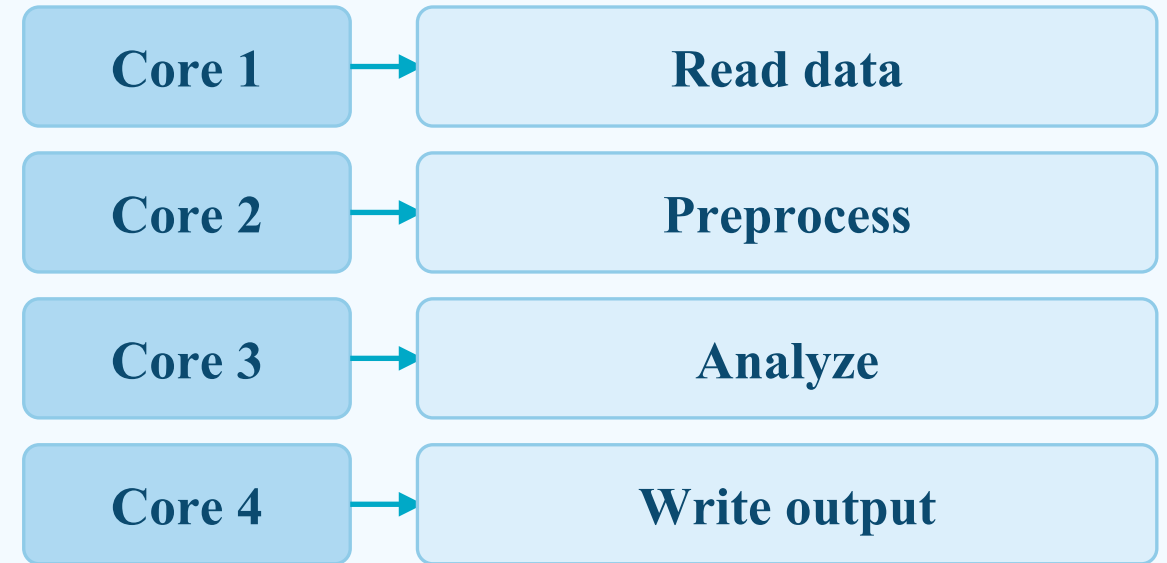
Task Parallelism

- Different tasks (possibly different operations) execute in parallel
- Each core performs a distinct role in a pipeline or workflow

Difference from Data Parallelism:

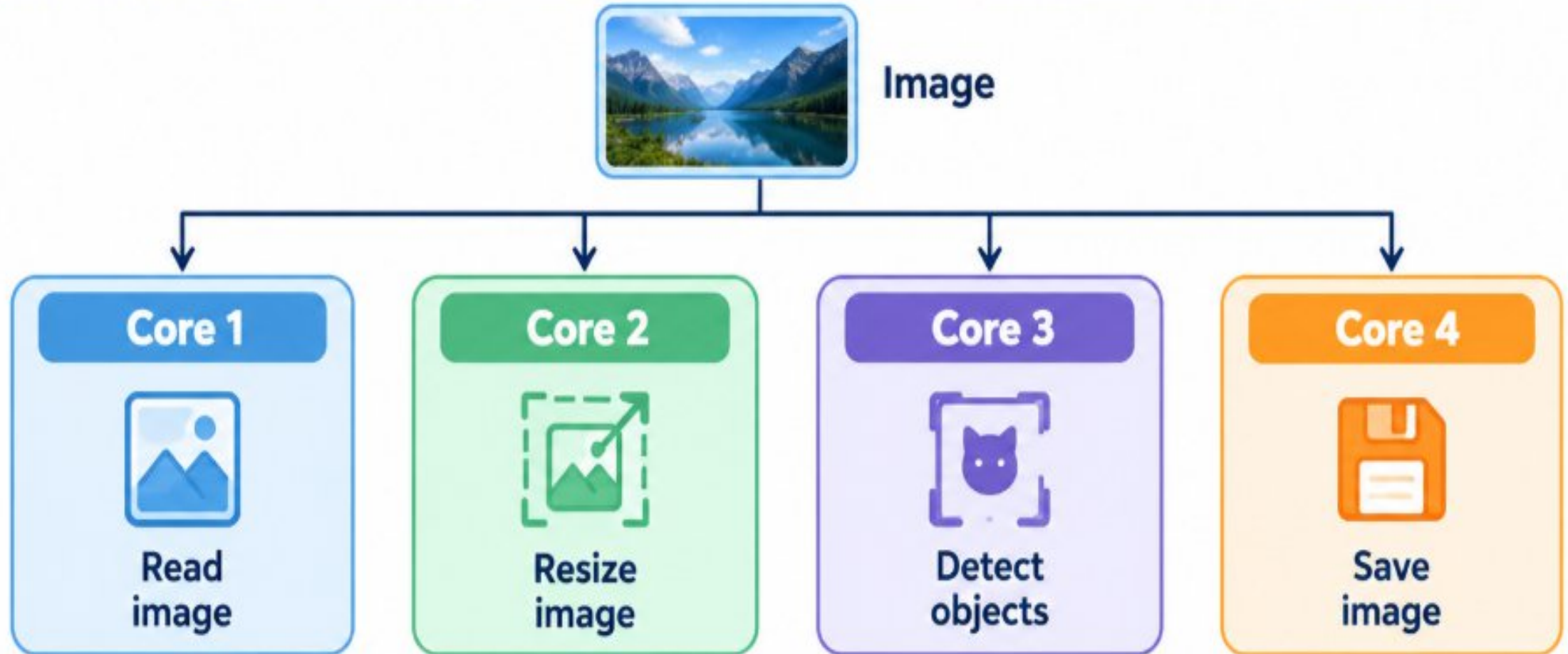
Data parallelism = same operation, different data.

Task parallelism = different operations, possibly on the same data.



Task Parallelism – Example (Image Processing)

Suppose a program receives an image and needs to perform several different operations:



Introduction to Flynn's Taxonomy

- Proposed by Michael Flynn (1966) to classify computer architectures
- Classification is based on two streams:
 - Instruction Stream — sequence of instructions executed
 - Data Stream — sequence of data items processed
- Combining single/multiple instruction streams with single/multiple data streams gives four categories

SISD

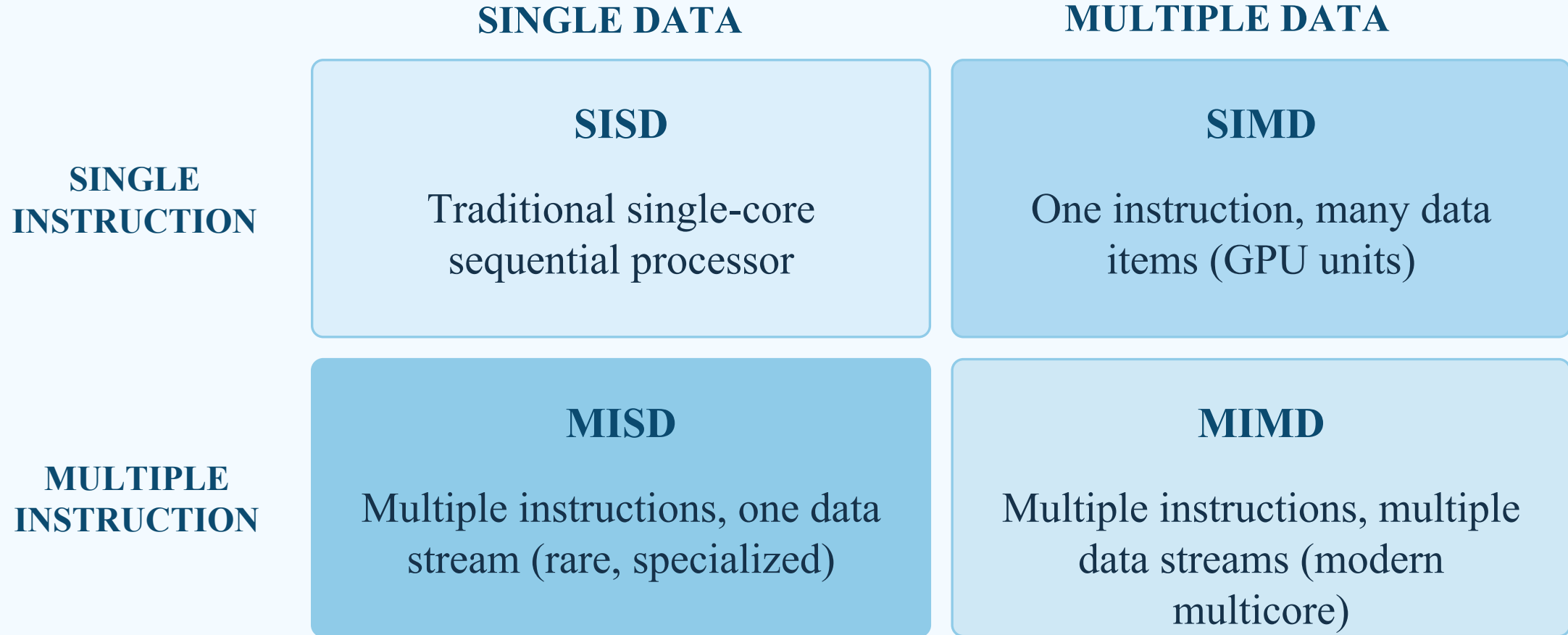
SIMD

MISD

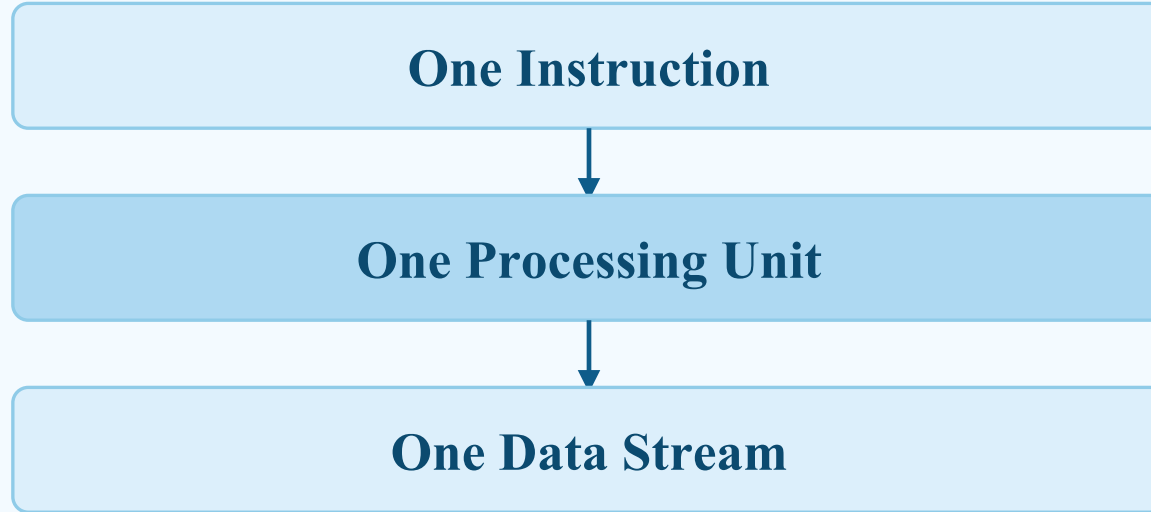
MIMD

KEY TAKEAWAY Flynn's taxonomy is a conceptual map — most real multicore CPUs are best described as MIMD.

Flynn's Taxonomy — The 2×2 Matrix



SISD — Single Instruction, Single Data



- **SISD – Single Instruction, Single Data**
One processor/core executes **one instruction at a time**.
- The instruction operates on **one data item at a time**.
- This represents **traditional sequential computing**.
- **No parallelism** — operations are performed one after another.

Example: Matrix Addition — SISD

Suppose:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

We want:

$$\mathbf{C} = \mathbf{A} + \mathbf{B}$$

In SISD, the processor performs **one addition at a time**:

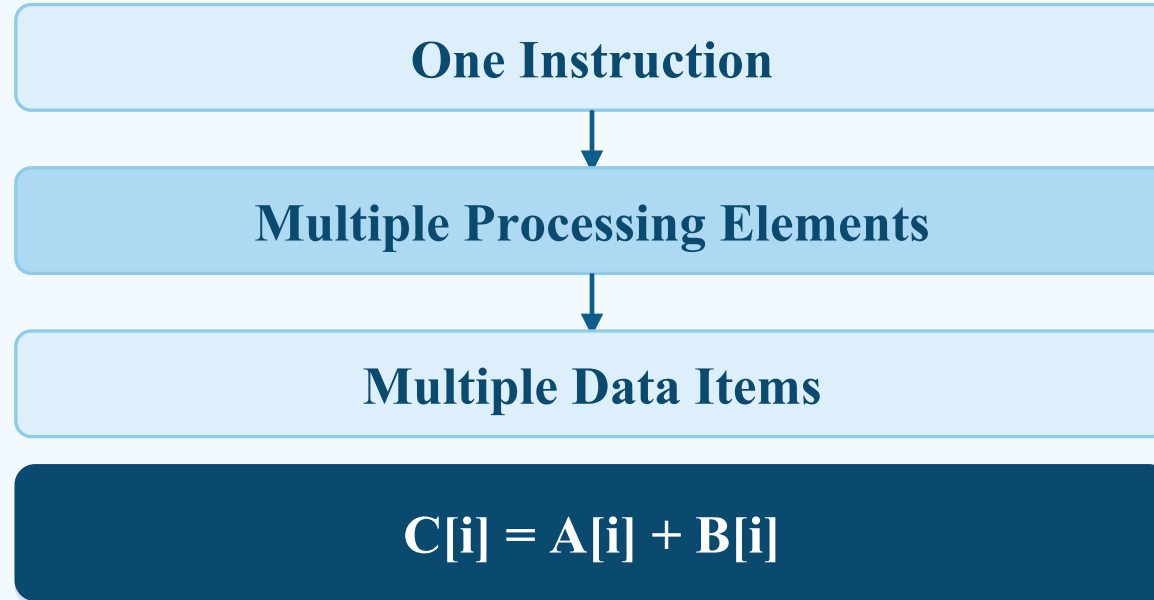
$$\text{Step 1} \rightarrow C[0][0] = A[0][0] + B[0][0] = 1 + 5 = 6$$

$$\text{Step 2} \rightarrow C[0][1] = A[0][1] + B[0][1] = 2 + 6 = 8$$

$$\text{Step 3} \rightarrow C[1][0] = A[1][0] + B[1][0] = 3 + 7 = 10$$

$$\text{Step 4} \rightarrow C[1][1] = A[1][1] + B[1][1] = 4 + 8 = 12$$

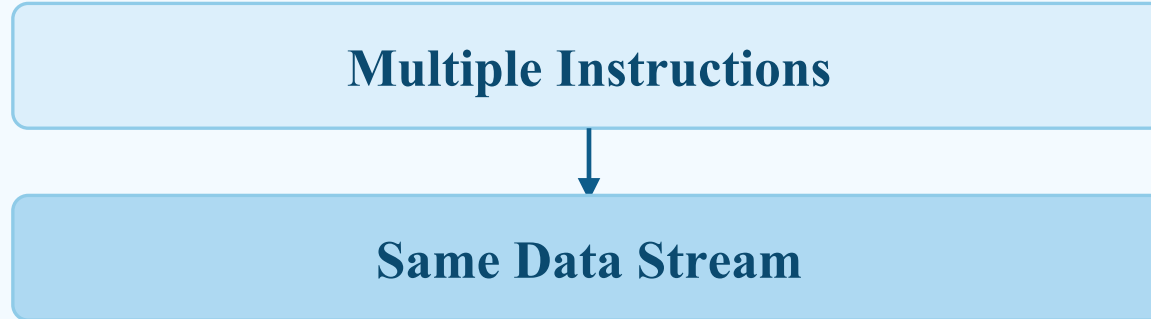
SIMD — Single Instruction, Multiple Data



- GPUs are strongly SIMD in nature
- Very efficient for data-parallel workloads like image processing.

The same 'add' instruction is applied to every pair of elements at once

MISD — Multiple Instruction, Single Data



- MISD is uncommon in general-purpose computing
- It is often discussed only as a conceptual/theoretical classification

Example:

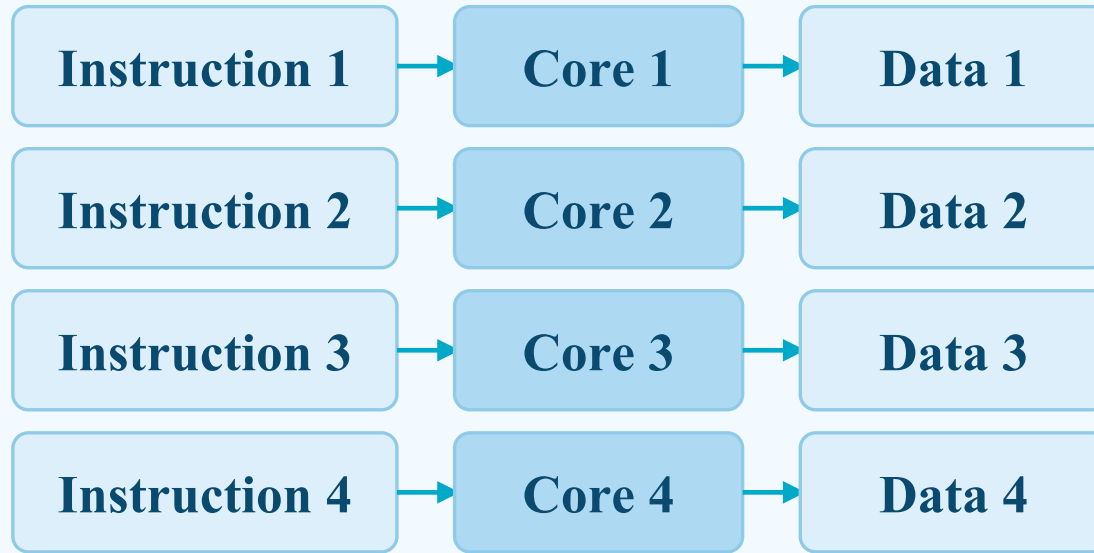
Suppose we have **one sensor reading**:

Temperature = 80°C

Different processing units perform **different operations on the same input**:

Processing Unit	Instruction	Same Data
Unit 1	Check whether temperature > 70°C	80°C
Unit 2	Convert Celsius → Fahrenheit	80°C
Unit 3	Calculate temperature × 2	80°C
Unit 4	Compare with another threshold	80°C

MIMD — Multiple Instruction, Multiple Data



- Each core executes its own instruction stream on its own data
- Modern multicore CPUs commonly fit MIMD — every core can run independent programs
- Most flexible and general category

Example: Image Processing — MIMD

Suppose we have 4 images:

Image 1 Image 2 Image 3 Image 4

We have 4 CPU cores:

Core	Instruction / Task	Data
Core 1	Resize	Image 1
Core 2	Convert to grayscale	Image 2
Core 3	Detect edges	Image 3
Core 4	Compress	Image 4

Flynn's Taxonomy — Comparison

	SISD	SIMD	MISD	MIMD
Instruction streams	Single	Single	Multiple	Multiple
Data streams	Single	Multiple	Single	Multiple
Execution	One op at a time	Same op, many data	Diff. ops, same data	Diff. ops, diff. data
Typical use	Traditional single core	GPU processing	Rare	General multicore
Example	Classic sequential CPU	GPU	Conceptual case	Multicore CPU

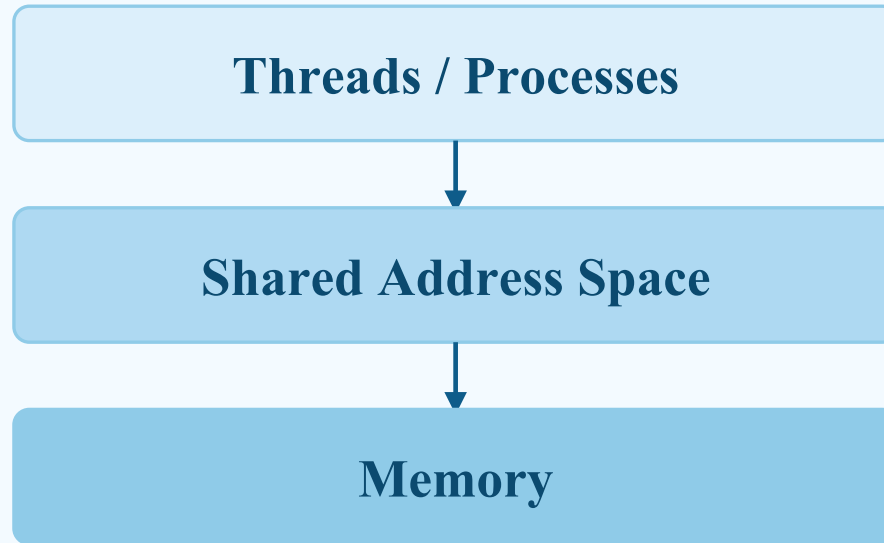
Models of Parallel Computation



1. Shared Memory Model

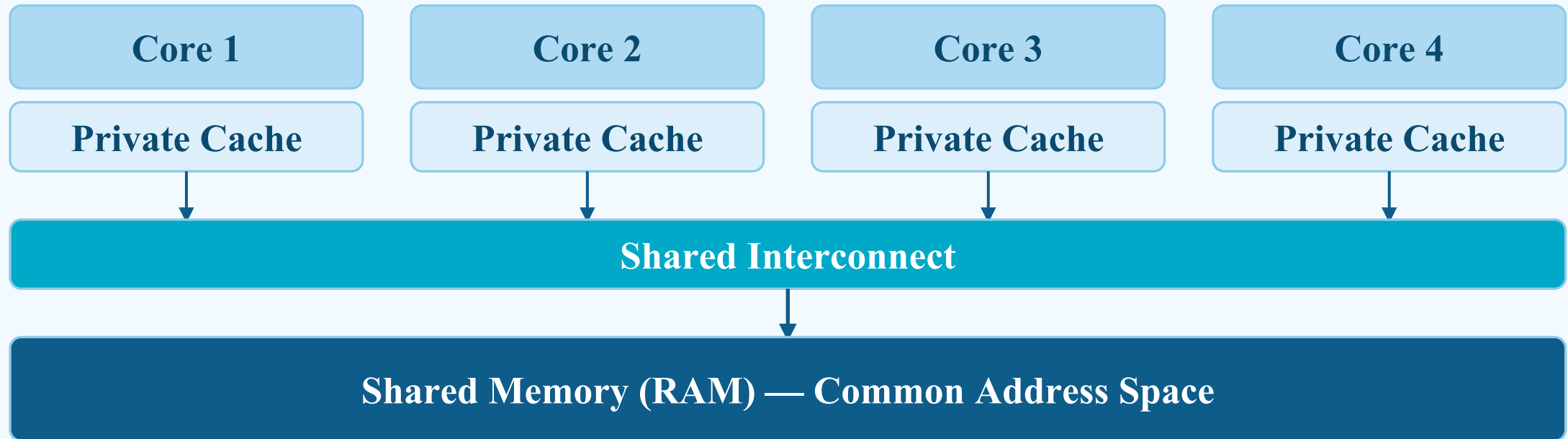
2. Distributed memory model

Shared-Memory Model



- Communication happens through shared variables / shared data structures
- All threads can directly read and write common memory
- Synchronization (locks, semaphores) become important to avoid conflicts

Shared-Memory Architecture



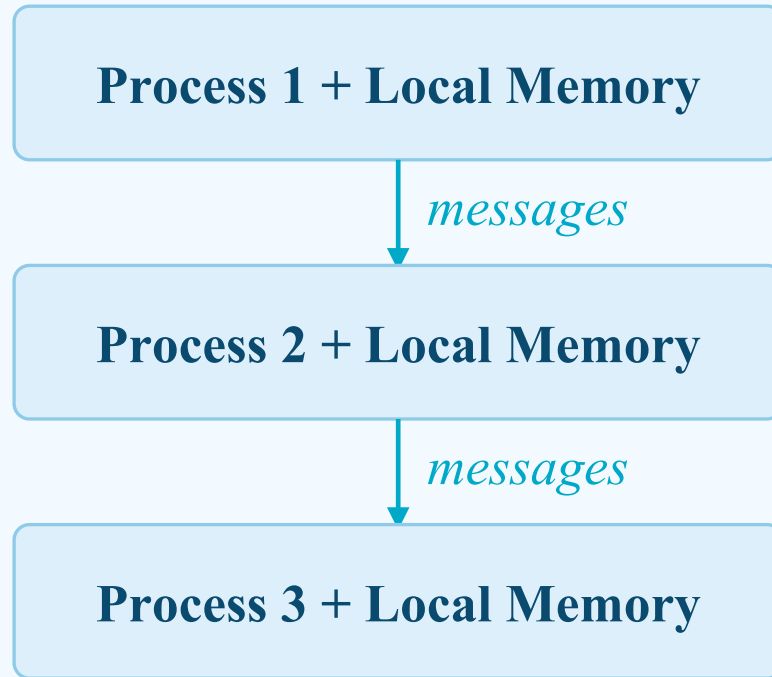
“All processors/cores can access a common address space.”

Communication between cores is easy (just read/write shared variables) — but synchronization become important challenges as more cores share the same memory.

How Does Cache Get Data?

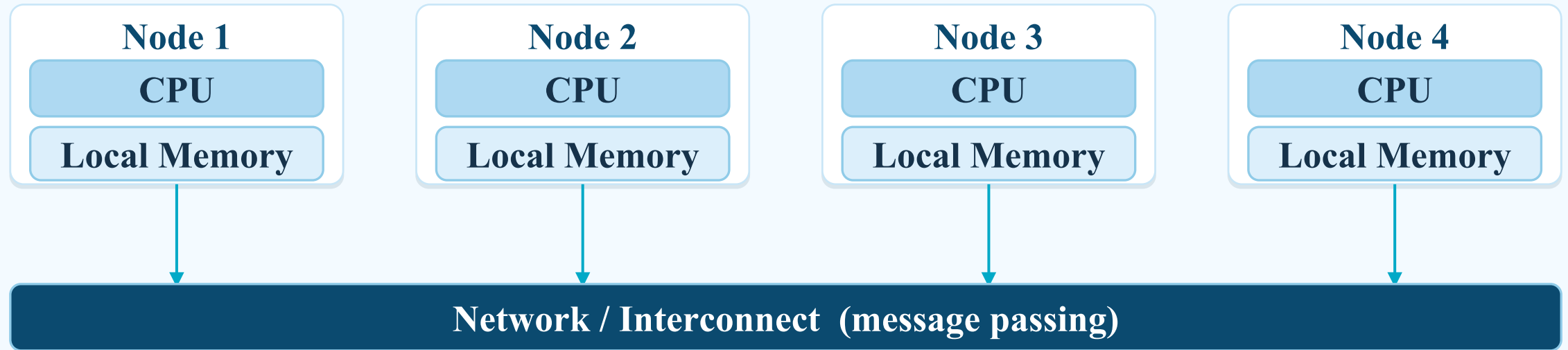
- Core 1 needs some data.
- Core 1 sends a request for the **location (memory address)** of that data to the cache.
- Cache checks: “**Do I have data from this location?**”
- **Yes** → **Cache Hit** → sends data to Core 1.
- **No** → **Cache Miss** → request goes toward RAM.
- RAM sends the data back, and it is placed in the cache.

Distributed-Memory / Message-Passing Model



- Each process has its own private, local memory
- No shared address space — one process cannot directly read another's memory
- Communication happens explicitly through message passing (e.g. MPI)

Distributed-Memory Architecture



Independent Nodes – Each node has its own **CPU** and **local memory**; memory is not directly shared between nodes.

Message Passing – Nodes communicate and exchange data through a **network/interconnect** by sending messages.

Parallel Execution – Different nodes can work on different parts of a problem **simultaneously**, making the architecture suitable for large-scale computing.

Shared vs Distributed Memory — Detailed Comparison

Aspect	Shared Memory	Distributed Memory
Memory	All processors share one memory	Each node has its own memory
Address Space	One common address space	Separate address space for each node
Communication	Through shared variables	Through message passing
Scalability	Suitable for fewer processors	Can support many nodes
Programming	Easier to program	More complex to program
Advantage	Fast communication between cores	Highly scalable
Disadvantage	Synchronization problems	Communication can take more time

Measuring Parallel Performance

Execution Time

- Total time taken to complete a computation

Speedup

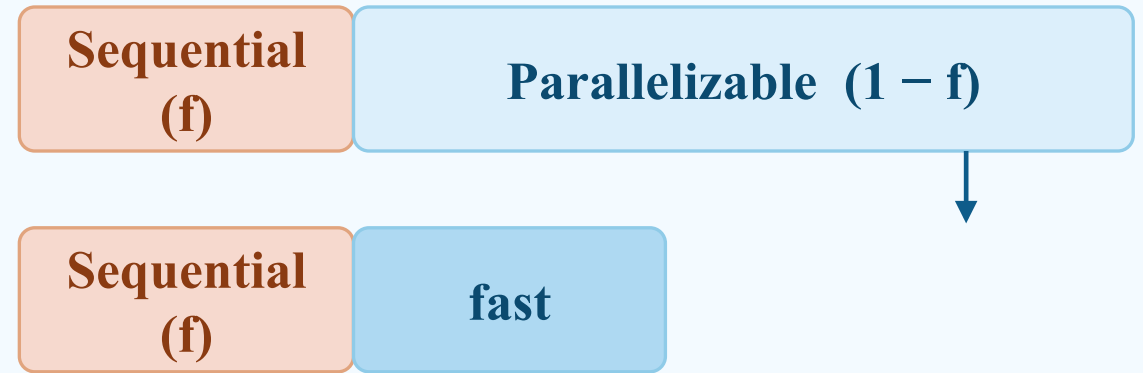
- How much faster the parallel version is compared to sequential

Scalability

- Whether performance keeps improving as more processors are added

Amdahl's Law: Motivation

- Every program has two parts:
 - A sequential portion — must run one step at a time
 - A parallelizable portion — can be split across processors
- Even if the parallel portion becomes infinitely fast, the sequential portion still takes the same time
- This sequential part limits the maximum possible speedup



Sequential part stays the SAME size even with more processors!

$$\text{Speedup} = \frac{\text{Execution Time on 1 Processor}}{\text{Execution Time on P Processors}}$$

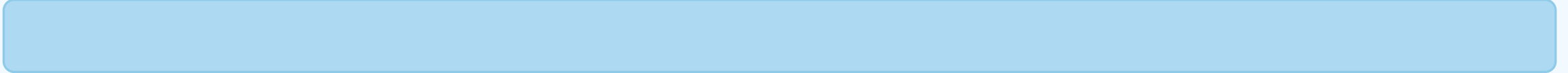
Amdahl's Law — Formula

$$S(p) = 1 / [f + (1 - f) / p]$$

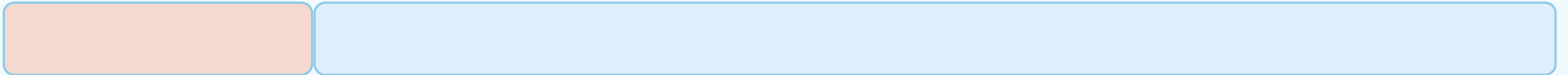
Symbol	Meaning
S(p)	Theoretical speedup with p processors
f	Fraction of the program that is strictly sequential
p	Number of processors/cores used
(1 - f)	Fraction of the program that can be parallelized

Amdahl's Law — Visualizing the Split

Step 1 — 100% on one processor



Step 2 — 20% sequential, 80% parallel



Step 3 — parallel part split across 4 processors



Step 4 — 20% still runs serially; total time set by (serial + fastest parallel chunk)



Amdahl's Law — Worked Example 1

Given

- $f = 0.3$ (30% sequential)
- $p = 8$ processors

Formula

$$S(p) = 1 / [f + (1-f)/p]$$

Substitution & Calculation

- $S(8) = 1 / [0.3 + 0.70/8]$
- $S(8) = 1 / [0.3 + 0.0875]$
- $S(8) = 1 / 0.3875$

$$S(8) \approx 2.58\times$$

Interpretation

With 30% of the program being sequential, using 8 processors gives a maximum theoretical speedup of about $2.58\times$ — far less than $8\times$, because the serial portion cannot be sped up.

Amdahl's Law — Worked Example 2

Given

- $f = 0.25$ (25% sequential)
- $p = 6$ processors

Formula

$$S(p) = 1 / [f + (1-f)/p]$$

Substitution & Calculation

- $S(6) = 1 / [0.25 + 0.75/6]$
- $S(6) = 1 / [0.25 + 0.1250]$
- $S(6) = 1 / 0.3750$

$$S(6) \approx 2.67\times$$

Interpretation

With 25% of the program being sequential, using 6 processors gives a maximum theoretical speedup of about $2.67\times$ — far less than $6\times$.

Amdahl's Law — Maximum Possible Speedup

$$\text{As } p \rightarrow \infty, S_{\text{max}} = 1 / f$$

- As the number of processors grows without bound, the parallel portion's time shrinks toward zero.

Example: $f = 0.20$

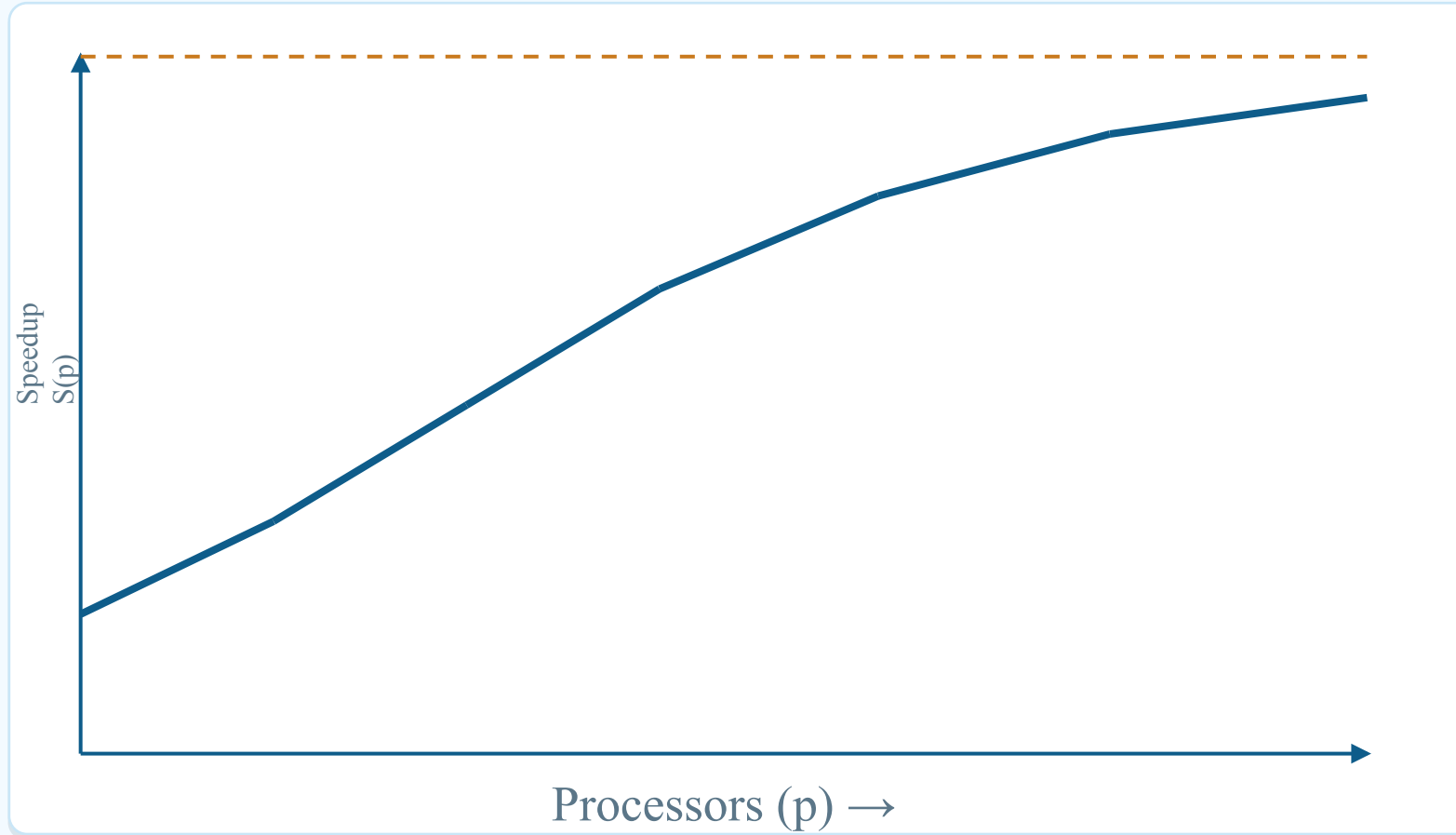
$$S_{\text{max}} = 1 / 0.20 = 5$$

Even with infinitely many processors, the maximum possible speedup is only 5×

Why this matters?

- This is one of the most frequently tested Amdahl concepts
- Users often mistakenly assume more processors always means proportionally more speedup
- Adding processors beyond the point of diminishing returns wastes resources

Amdahl's Law — Speedup Curve



- Speedup rises quickly at first
- Curve flattens as p grows — diminishing returns

Gustafson's Law — Motivation

- **Amdahl's Law:** Problem size stays **the same** when we add more processors.
- But in real life, if we get **more processors**, we often try to solve **bigger problems**.
- **Gustafson's Law** considers this **growing problem size**.
- It tells us how much **larger a problem we can solve in roughly the same time** by using more processors.

Amdahl's Law

Fixed problem size

*“How much faster can I solve **THIS** problem using more processors?”*

Gustafson's Law

Scaled problem size

*“How much **BIGGER** a problem can I solve in the same time?”*

Gustafson's Law — Formula

$$S_G(p) = p - f(p - 1)$$

Symbol	Meaning
$S_G(p)$	Scaled speedup with p processors
f	fraction of the workload
p	Number of processors used

Gustafson's Law — Growing the Problem

1 processor
problem size
 $\times 1.0$

4 processors
problem size
 $\times 1.8$

8 processors
problem size $\times 2.6$

*Roughly the same total time — but a larger problem solved
within it as p grows*

KEY TAKEAWAY Scalability means using more processors to solve a larger problem in about the same amount of time.

Gustafson's Law — Worked Example

Given

- $f = 0.10$ (10% sequential)
- $p = 16$ processors
- Original workload = 100 tasks , 100 sec, 1 core.

Formula

$$S_G(p) = p - f(p-1)$$

Substitution & Calculation

- $S_G(16) = 16 - 0.10 \times (16-1)$
- $S_G(16) = 16 - 0.10 \times 15$
- $S_G(16) = 16 - 1.5$

$$S_G(16) = 14.5\times$$

Interpretation

With only 10% of the workload being sequential, scaling up to 16 processors lets us solve a problem about $14.5\times$ larger in roughly the same time.

Scaled workload = Original workload $\times$ Gustafson speedup

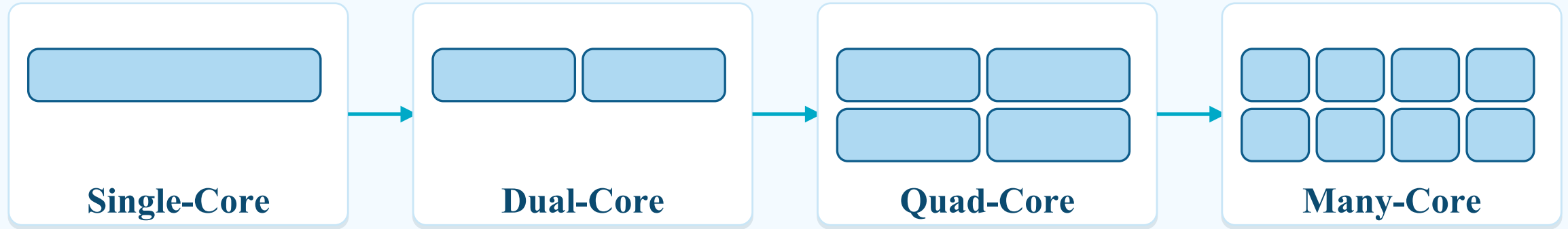
$$\begin{aligned} &= 100 \times 14.5 \\ &= 1450 \text{ units} \end{aligned}$$

Amdahl's Law vs Gustafson's Law

Aspect	Amdahl's Law	Gustafson's Law
Problem size	Fixed	Scaled with processor count
Focus	Maximum possible speedup	Scalability for larger workloads
Key insight	Serial fraction limits speedup	More processors solve bigger problems well
Formula	$S(p) = 1 / [f + (1-f)/p]$	$S_G(p) = p - f(p-1)$
Best used when...	Problem size is truly fixed	Workload naturally grows with resources

KEY TAKEAWAY Both laws are correct — they simply answer different questions about scaling a workload.

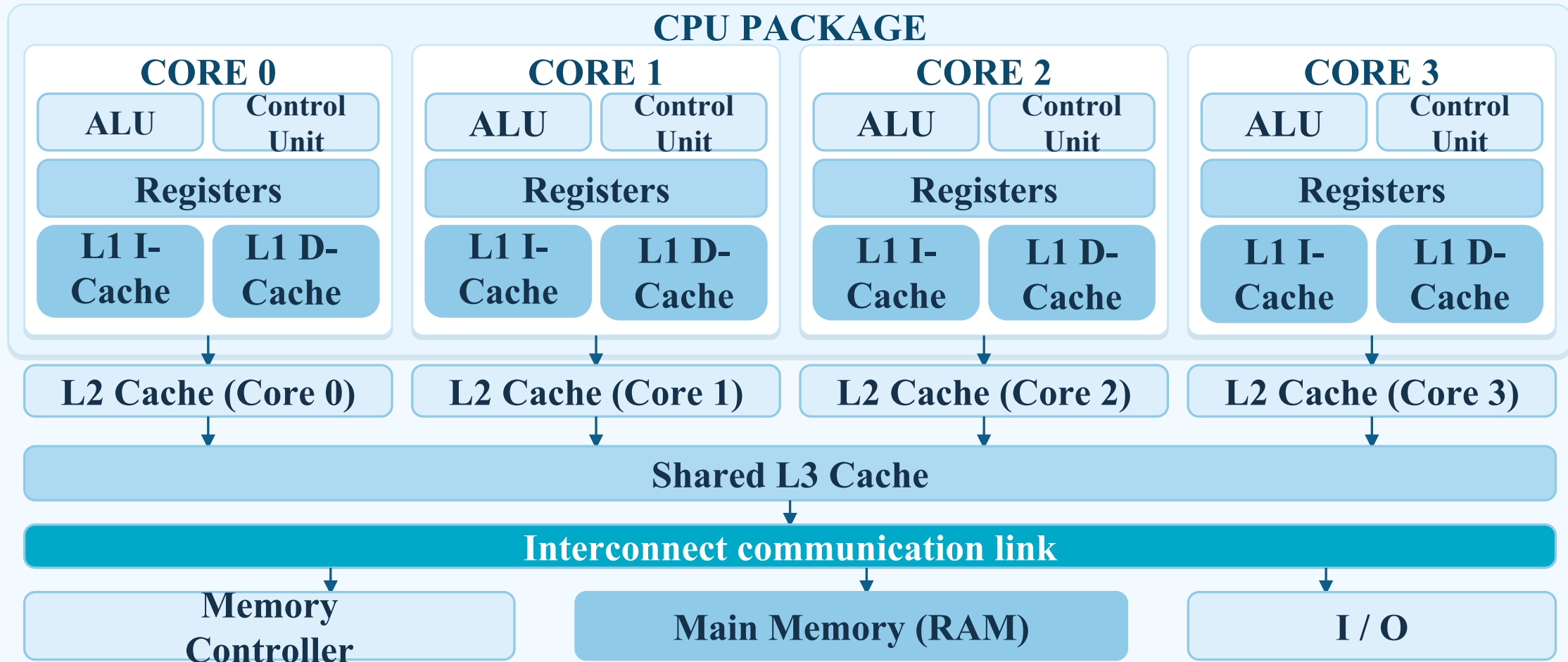
Evolution of Multicore Processors



KEY TAKEAWAY Single-core performance hit power and heat limits, so using more cores became the better way to increase performance.

- Multiple simple cores can work together to give better performance than one very powerful core.
- Programs must be designed for parallel execution to make full use of multiple cores.

Modern Multicore CPU Architecture



ALU: does arithmetic/logic • Control Unit: directs instruction execution • Registers: fastest storage • L1/L2/L3: progressively larger, shared caches • Interconnect: links cores to memory • RAM: main system memory

What Happens When a Core Executes a Program?

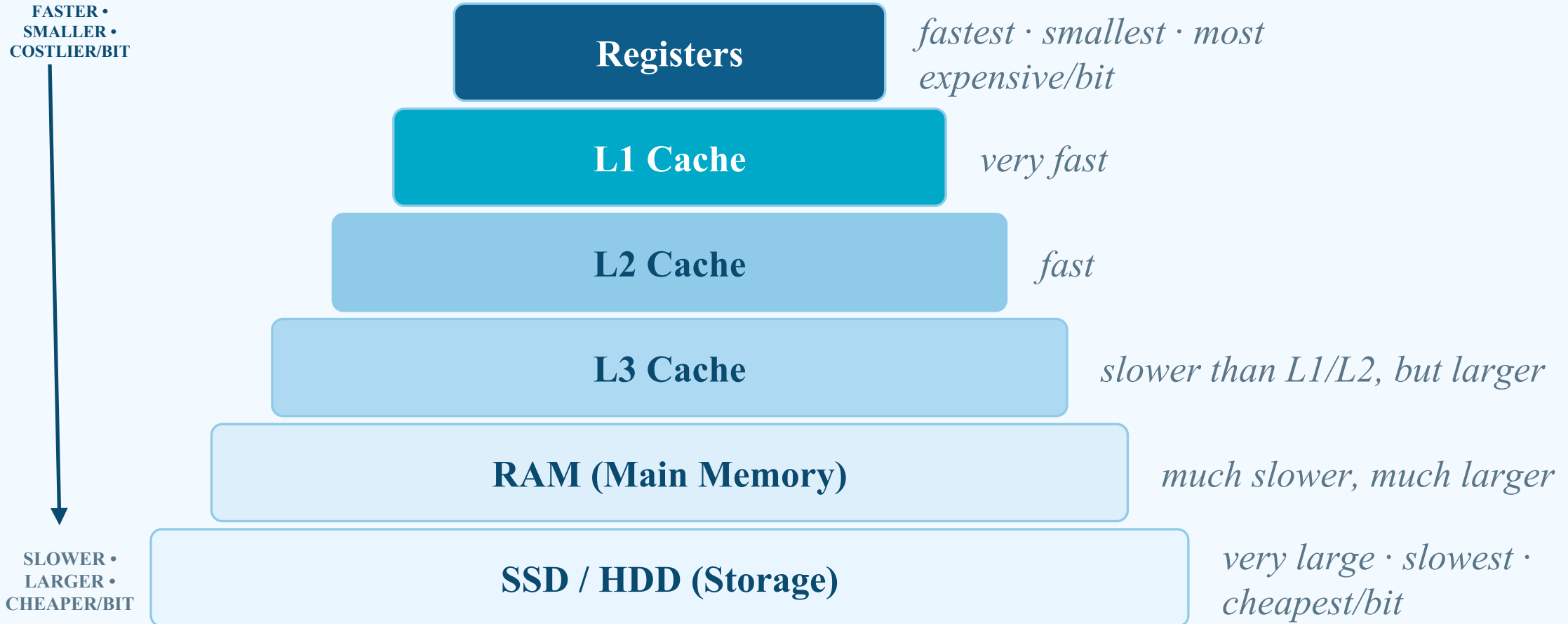
Instruction



Where memory levels fit into this pipeline

- Fetch: the next instruction is retrieved, typically from L1 Instruction Cache
- Decode / Execute: uses Registers (fastest storage, right inside the core) for operands
- Memory Access: data reads/writes go through L1 Data Cache, then L2 and shared L3 if not found (a cache miss escalates further, ultimately to RAM)
- Write Back: results are written back to registers or memory

Memory Hierarchy



Note: exact sizes and latencies vary across processors — this shows the general trend, not universal figures.

Cache Organization

What is cache?

- Small, fast memory that stores frequently/recently used data close to the core

Why it exists

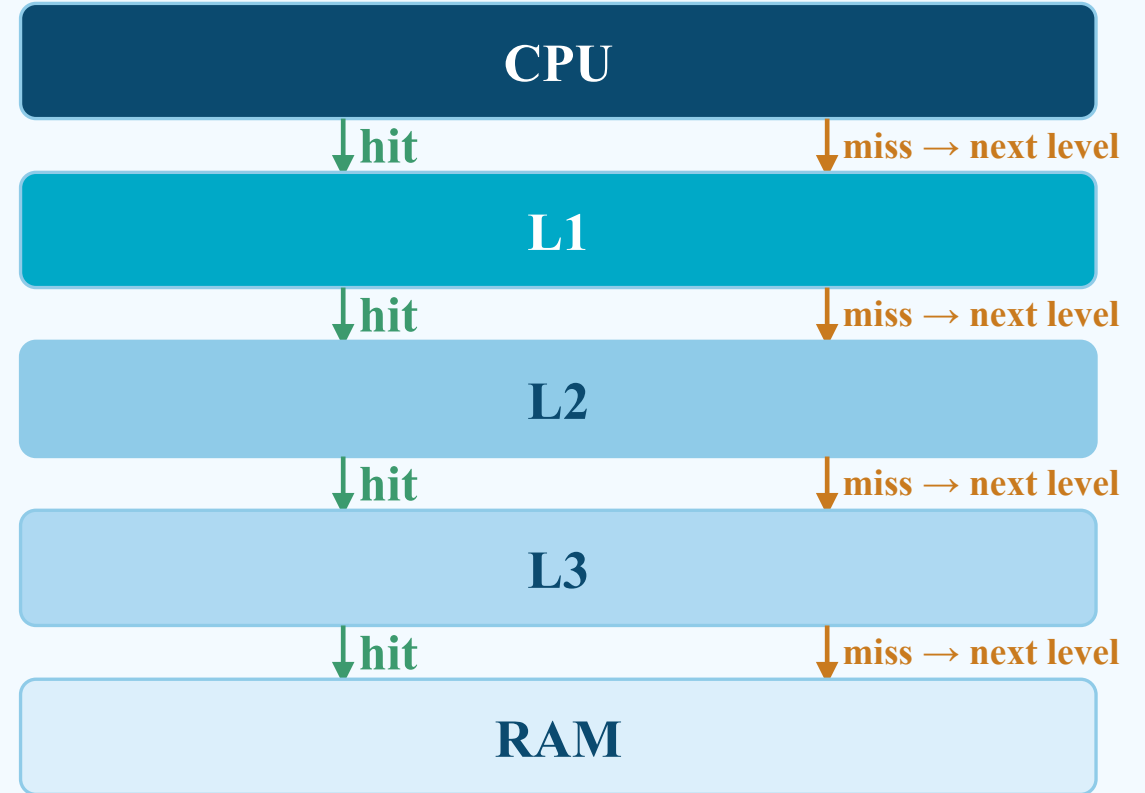
- Bridges the huge speed gap between the core and main memory (RAM)

Locality principles

- Temporal locality: recently used data is likely reused soon
- Spatial locality: nearby data is likely to be used soon

Cache line / block

- Data is moved between cache and memory in fixed-size chunks (cache lines)



Cache Performance

Cache Hit

Requested data is found in the cache — fast access, no need to go to slower memory.

- Hit rate: fraction of accesses that are hits
- Miss rate: fraction of accesses that are misses.
- Cache latency means the time taken by the CPU to access data from the cache.
- Memory access time: time to access main memory on a miss
- Locality: programs with good locality achieve higher hit rates

Cache Miss

Requested data is not in the cache — must be fetched from a lower (slower) memory level.

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

AMAT = Average Memory Access Time. Even a small miss rate matters a lot, because the miss penalty (time to reach RAM) is very large compared to a cache hit.

A processor has a cache access time of **2 ns**. Due to some cache misses, the miss rate is **10%**, and each miss adds **50 ns** of delay. **What is the average time taken by the processor to access the required data?**

$$\begin{aligned}\text{AMAT} &= \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty} \\ &= 2 + (0.10 \times 50) \\ &= 7 \text{ ns}\end{aligned}$$

Miss penalty → extra time needed when the data is **not** in the cache and must be fetched from a lower level/RAM.

Cache Mapping Techniques

- **Cache mapping** determines where a block of data from **main memory (RAM)** can be placed in the cache.
- Since cache is much smaller than RAM, **not every memory block can occupy any location.**
- A mapping technique defines the relationship between: **Memory Blocks → Cache Lines**

Why is mapping needed?

Suppose:

- RAM has **1000 blocks**
- Cache has only **100 lines**

We need a method to decide:

“If block 250 comes from RAM, where should it be placed in the cache?”

The three common techniques are:

- 1. Direct Mapping**
- 2. Fully Associative Mapping**
- 3. Set-Associative Mapping**

1. Direct Mapping:

In **direct mapping**, each memory block can be placed in **only one specific cache line**.

Basic idea:

Cache Line = Memory Block Number mod Number of Cache Lines

Direct Mapping —Example

Given:

- Cache = **4 lines**
- Memory blocks = **0, 1, 2, 3, 4, 5...**

Formula:

Cache Line = Memory Block mod 4

Memory Block	Cache Line
0	0
1	1
4	0
5	1

$0 \bmod 4 = 0 \rightarrow \text{Line 0}$

$1 \bmod 4 = 1 \rightarrow \text{Line 1}$

$2 \bmod 4 = 2 \rightarrow \text{Line 2}$

$3 \bmod 4 = 3 \rightarrow \text{Line 3}$

$4 \bmod 4 = 0 \rightarrow \text{Line 0}$

$5 \bmod 4 = 1 \rightarrow \text{Line 1}$

Advantages

- Simple to implement
- Low hardware complexity

Disadvantage

- Different memory blocks may compete for the same cache line.

2. Fully Associative Mapping

In fully associative mapping, a memory block can be placed in **any cache line**.
There is **no fixed cache line** for a particular memory block.

Example:

Suppose cache has 4 lines:

Cache:

Line 0

Line 1

Line 2

Line 3

Memory block 25 can go into:

Line 0 OR Line 1 OR Line 2 OR Line 3

The cache can choose **any available line**.

Why is this useful?

Suppose:

Block 0 → Line 0

Block 4 → Line 1

Block 8 → Line 2

Block 12 → Line 3

Unlike direct mapping, these blocks don't have to compete for one predetermined line.

Advantage

- Flexible placement
- Fewer conflict misses

Disadvantages

- More complex hardware
- Searching for a block is more expensive

3. Set-Associative Mapping

Set-associative mapping combines the ideas of:

Direct Mapping + Fully Associative Mapping

The cache is divided into **sets**, and each set contains multiple cache lines called **ways**. A memory block is mapped to **one particular set**, but within that set it can occupy **any available way**.

Example: 2-way Set Associative Cache

Suppose:

- Cache has **4 lines**
- We divide it into **2 sets**
- Each set has **2 lines (2 ways)**

Set 0 → Line 0 | Line 1

Set 1 → Line 2 | Line 3

A memory block first determines its **set**.
Then it can occupy either line within that set.

Example

If:

Block 0 $\rightarrow$ Set 0

Block 4 $\rightarrow$ Set 0

Block 8 $\rightarrow$ Set 0

The blocks mapped to Set 0 can use its available ways.

Advantages

- Less complex than fully associative mapping
- Good balance between performance and hardware complexity.

Disadvantages

- **More complex than direct mapping** because the cache is divided into sets and ways.
- **More costly to implement** than direct mapping.

Comparison of Mapping Techniques

Feature	Direct	Fully Associative	Set-Associative
Placement	One fixed line	Any line	Any line within a set
Flexibility	Low	High	Medium
Hardware complexity	Low	High	Medium
Conflict misses	More	Fewer	Fewer than direct

Performance Considerations in Parallel Computing

1. **Execution Time** – Parallel computing should reduce the time required to complete a task.
2. **Speedup** – Measures how much faster the parallel program is compared to the sequential program.
3. **Efficiency** – Measures how effectively the available processors/cores are being used.
4. **Load Balancing** – Work should be distributed equally among processors to avoid some processors becoming idle.
5. **Communication Overhead** – Communication between processors can add extra time and reduce performance.
6. **Synchronization Overhead** – Waiting for other processors or coordinating shared data can reduce performance.
7. **Scalability** – A parallel program should continue to perform well when more processors are added.
8. **Memory Performance** – Memory access speed and data movement can become a major performance limitation.